Diffusion Seminar (Week5)

SDE, ODE/ Score-based Modeling

Woojin Kim

AIDAS LAB
Seoul National University
wjk9904@snu.ac.kr

AIDAS Lab

Table of contents

1 Recap: DDPM

2 DDIM: Denoising Diffusion Implicit Models

3 Score-based Modeling: Diffusion as SDE/ODE

Recap: DDPM
(Denoising Diffusion Probabilistic Models)

DDPM: Denoising Diffusion Probabilistic Models

- Objective

  To learn a generative process that reverses a fixed noising process, progressively denoising pure noise into realistic data samples.

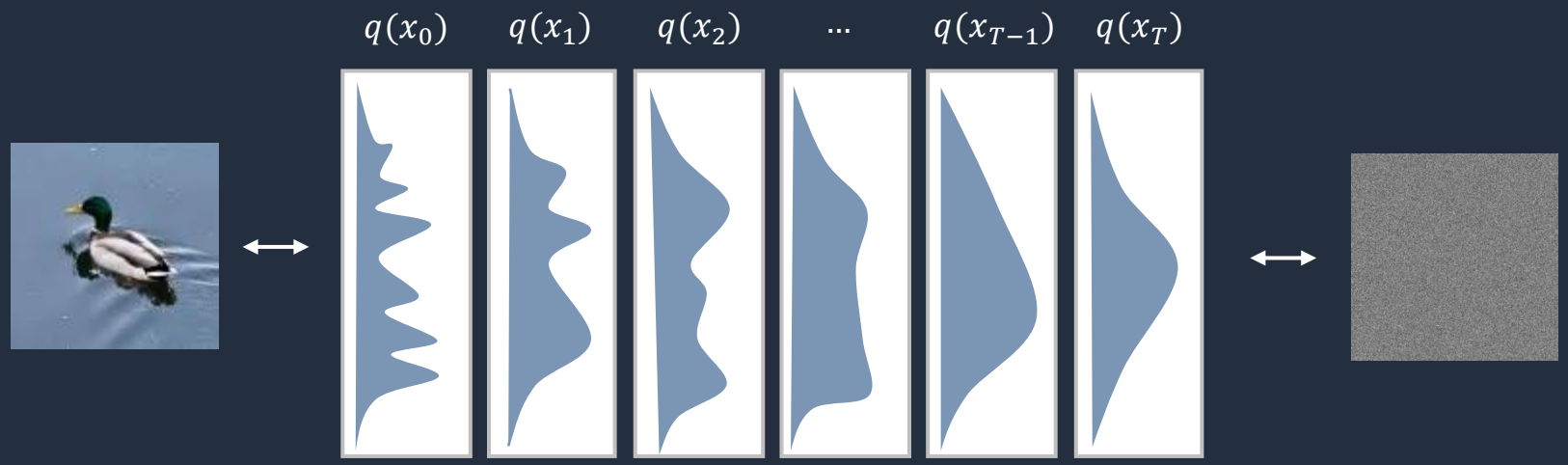
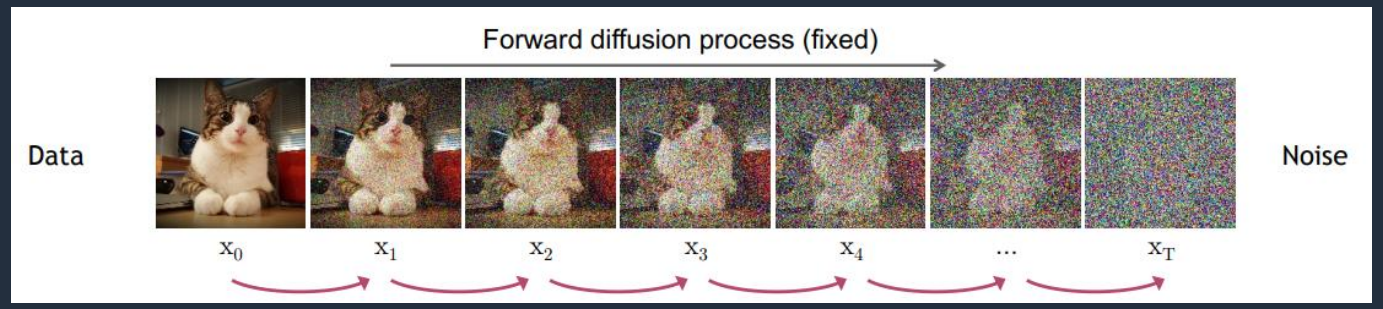


Image Credit: Jihwan Hong

DDPM: Denoising Diffusion Probabilistic Models

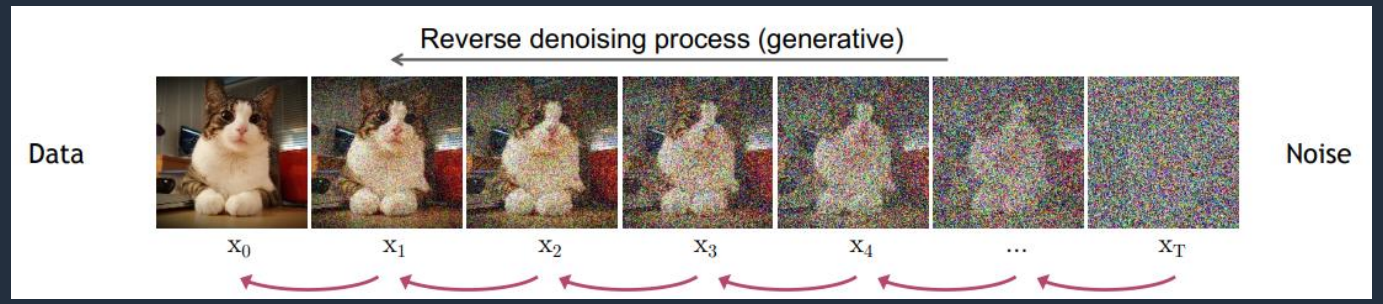
- Forward (noising) process



$$q(x_{1:T}|x_0) := \prod_{t=1}^{T} q(x_t|x_{t-1})$$

$$q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t \mathrm{I})$$

- Backward (denoising) process



$$p_\theta(\boldsymbol{x}_{0:T}) := p(\boldsymbol{x}_T) \prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$$

$$p(\boldsymbol{x}_T) := \mathcal{N}(\boldsymbol{x}_T; \mathbf{0}, \mathbf{I})$$

$$p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) := \mathcal{N}(\boldsymbol{x}_{t-1}, \boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t), \boldsymbol{\Sigma}_\theta(\boldsymbol{x}_t, t))$$

Image Credit: Copyright (c) 2023 cvpr2023-tutorial-diffusion-models (MIT license)

DDPM: Denoising Diffusion Probabilistic Models

- Loss function

Original Loss: $$\mathbb{E}_{x_0,\, \epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I}),\, \mathbf{t}}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\,||\,(\boldsymbol{\epsilon}-\boldsymbol{\epsilon}_\theta(\sqrt{\overline{\alpha_t}}\boldsymbol{x}_0+\sqrt{(1-\overline{\alpha_t})}\boldsymbol{\epsilon},t))||^2\right]$$

Simple Loss: $$\mathbb{E}_{x_0,\, \epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I}),\, \mathbf{t}}\left[||\,(\boldsymbol{\epsilon}-\boldsymbol{\epsilon}_\theta(\sqrt{\overline{\alpha_t}}\boldsymbol{x}_0+\sqrt{(1-\overline{\alpha_t})}\boldsymbol{\epsilon},t))||^2\right]$$

Discard the weighting term

- Sampling

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Ref: Ho et al., "Denoising Diffusion Probabilistic Models", NeurIPS 2020.

DDPM: Denoising Diffusion Probabilistic Models

- Notable Properties

  a. Markovian forward/backward process

$$q(x_{1:T}|x_0) \coloneqq \prod_{t=1}^{T} q(x_t|x_{t-1}) \qquad p_\theta(\boldsymbol{x}_{0:T}) \coloneqq p(\boldsymbol{x}_T) \prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$$

  b. Variance preserving forward pass

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) \coloneqq \mathcal{N}(\boldsymbol{x}_t; \sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t\mathbf{I}) \longrightarrow \mathrm{Var}(\boldsymbol{x}_{t-1}) = \mathbf{I}, \quad \mathrm{Var}(\boldsymbol{x}_t) = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{I} = \mathbf{I}$$

  c. Arbitrary Timesteps at Training

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) \coloneqq \mathcal{N}(\boldsymbol{x}_t; \sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t\mathbf{I}) \longrightarrow q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}\big(\boldsymbol{x}_t; \sqrt{\overline{\alpha_t}}\boldsymbol{x}_0, (1-\overline{\alpha_t})\mathbf{I}\big)$$

DDIM: Denoising Diffusion Implicit Models

Generative Learning Trilemma

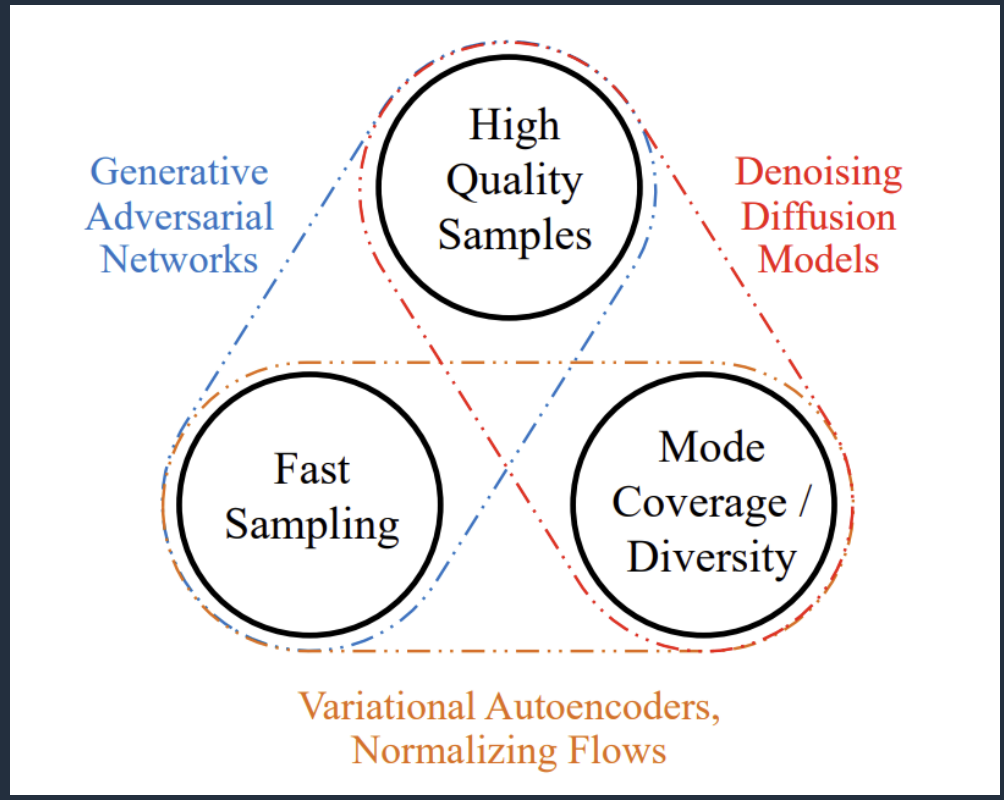
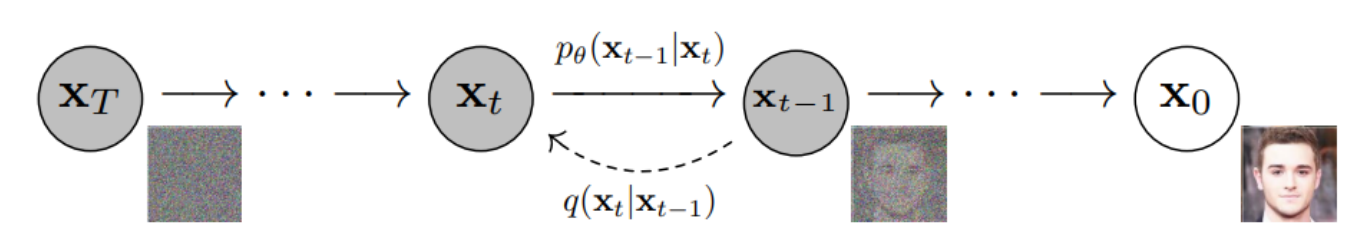


Image credit: Xiao et al., "Tackling the generative learning trilemma with denoising diffusion gans", ICLR 2022.

DDPM sampling is slow



$$q(x_{1:T}|x_0) \coloneqq \prod_{t=1}^{T} q(x_t|x_{t-1})$$

$$p_\theta(\boldsymbol{x}_{0:T}) \coloneqq p(\boldsymbol{x}_T) \prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$$

- Forward/Reverse is both Markovian
- Requires many iterations (~1000 steps)
  - To arrive at pure noise, a sufficient amount of noise must be injected
  - If noise per step is too high, Gaussian approximation of the reverse posterior breaks

→ **Can we make this faster without retraining?**

Ref: Ho et al., "Denoising Diffusion Probabilistic Models", NeurIPS 2020.

Image credit: Xiao et al., "Tackling the generative learning trilemma with denoising diffusion gans", ICLR 2022.

Matching the Marginal

- Recall

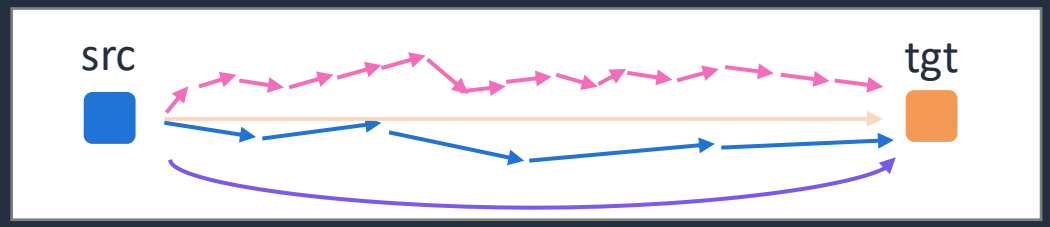
$$\boldsymbol{L_r} = \mathbb{E}_{\boldsymbol{x_0},\, \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I}),\, \mathbf{t}}\left[|| \left(\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\underbrace{\sqrt{\overline{\alpha_t}}\boldsymbol{x}_0 + \sqrt{(1-\overline{\alpha_t})}\boldsymbol{\epsilon}}_{\boldsymbol{x_t}\ \ where\ x_t\sim q(x_t|x_0)}, t)\right)||^2\right]$$

- Loss term only depends on the **marginal**, not the **joint**

Marginal: $q(x_t|x_0)$

Joint: $q(x_{1:t}|x_0)$

There exists many different inference trajectories that share the same marginal



Then, which trajectory should we sample from?

Non-Markovian Forward Process

- Assume there exists a family Q of inference distributions

$$q_\sigma(x_{1:t}|x_0) \coloneqq q_\sigma(x_T|x_0)\prod_{t=2}^{T} q_\sigma(x_{t-1}|x_t, x_0)$$

- Marginal: $q_\sigma(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\sqrt{\alpha_t}\boldsymbol{x}_0, (1-\alpha_t)\mathbf{I})$
- Let posterior: $q_\sigma(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\left(\sqrt{\alpha_{t-1}}\boldsymbol{x}_0 + \sqrt{1-\alpha_{t-1}-{\sigma_t}^2}\cdot\frac{x_t-\sqrt{\alpha_t}x_0}{\sqrt{1-\alpha_t}}, \boldsymbol{\sigma_t^2 I}\right)$

This leads to the above marginal (proof with induction)

- Forward process: $q_\sigma(\boldsymbol{x}_t|x_{t-1}, \boldsymbol{x}_0) = \frac{q_\sigma(x_{t-1}|x_t,x_0)q_\sigma(x_t|x_0)}{q_\sigma(x_{t-1}|x_0)}$

-> This is non-markovian since it depends on both x_{t-1}, x_0

DDPM transition

$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$
$\coloneqq \mathcal{N}(\boldsymbol{x}_t; \sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t\mathbf{I})$

Notation

$\alpha_t(DDIM) = \overline{\alpha_t}\ in\ DDPM$

Ref: Song et al., "Denoising Diffusion Implicit Models", ICLR 2021.

Generative Process

- Then, what should the generative process be?

  Posterior: $q_\sigma(\boldsymbol{x_{t-1}}|\boldsymbol{x_t}, \boldsymbol{x_0}) = \mathcal{N}\left(\sqrt{\alpha_{t-1}}\boldsymbol{x_0} + \sqrt{1-\alpha_{t-1}-{\sigma_t}^2}\cdot\frac{x_t-\sqrt{\alpha_t}x_0}{\sqrt{1-\alpha_t}}, \boldsymbol{\sigma_t^2 I}\right)$

- x_t is known, first find x_0

  $$Predicted\ x_0 \coloneqq f_\theta(x_t) \coloneqq (x_t - \sqrt{1-\alpha_t}\cdot\epsilon_\theta(x_t))/\sqrt{\alpha_t}$$

Recall

$$\boldsymbol{x_t} = \sqrt{\alpha_t}\boldsymbol{x_0} + \sqrt{(1-\alpha_t)}\boldsymbol{\epsilon_t}$$

- Generative process from $p_\theta(x_T) = N\ (0, I)$

$$p_\theta(x_{t-1}|x_t) = \begin{cases} N(f_\theta(x_1), \sigma_1^2 I) & if\ t=1 \\ q_\sigma(x_{t-1}|x_t, f_\theta(x_t)) & otherwise \end{cases}$$

- Training objective $J_\sigma(\epsilon_\theta)$ is almost same as the one in DDPM simplified objective (L_{simple}) (For detailed proof, see the paper)

Ref: Song et al., "Denoising Diffusion Implicit Models", ICLR 2021.

Denoising Diffusion Implicit Models

Posterior: $q_\sigma(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\left(\sqrt{\alpha_{t-1}}\boldsymbol{x}_0 + \sqrt{1-\alpha_{t-1}-{\sigma_t}^2}\cdot\frac{x_t-\sqrt{\alpha_t}x_0}{\sqrt{1-\alpha_t}}, \boldsymbol{\sigma_t^2 I}\right)$

Transition: $p_\theta(x_{t-1}|x_t) = \begin{cases} N(f_\theta(x_1), \sigma_1^2 I) & if\ t=1 \\ q_\sigma(x_{t-1}|x_t, f_\theta(x_t)) & otherwise \end{cases}$

Sampled x_{t-1}: $x_{t-1} = \sqrt{\alpha_{t-1}}\left(\frac{x_t-\sqrt{1-\alpha_t}\epsilon_\theta(x_t)}{\sqrt{\alpha_t}}\right) + \sqrt{1-\alpha_{t-1}-\sigma_t^2}\epsilon_\theta(x_t) + \sigma_t\epsilon_t$

Predicted x_0 — Direction toward x_t

$* d_t = x_t - \sqrt{\alpha_t}x_0 = \sqrt{1-\alpha_t}\epsilon_t$

Ref: Song et al., "Denoising Diffusion Implicit Models", ICLR 2021.

Comparison with DDPM

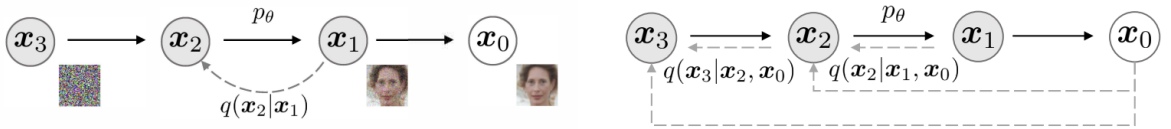


Figure 1: Graphical models for diffusion (left) and non-Markovian (right) inference models.

Sampled x_{t-1}:

$$x_{t-1} = \sqrt{\alpha_{t-1}}\left(\frac{x_t - \sqrt{1-\alpha_t}\epsilon_\theta(x_t)}{\sqrt{\alpha_t}}\right) + \sqrt{1-\alpha_{t-1}-\sigma_t^2}\epsilon_\theta(x_t) + \sigma_t\epsilon_t$$

When $\sigma_t = 0$, forward process becomes deterministic (not stochastic)

$$x_{t-1} = \sqrt{\alpha_{t-1}}\left(\frac{x_t - \sqrt{1-\alpha_t}\epsilon_\theta(x_t)}{\sqrt{\alpha_t}}\right) + \sqrt{1-\alpha_{t-1}}\epsilon_\theta(x_t)$$

When $\sigma_t = \sqrt{(1-\alpha_{t-1})/(1-\alpha_t)}\sqrt{1-\alpha_t/\alpha_{t-1}}$, process is same as DDPM

DDPM sampling

Algorithm 2 Sampling

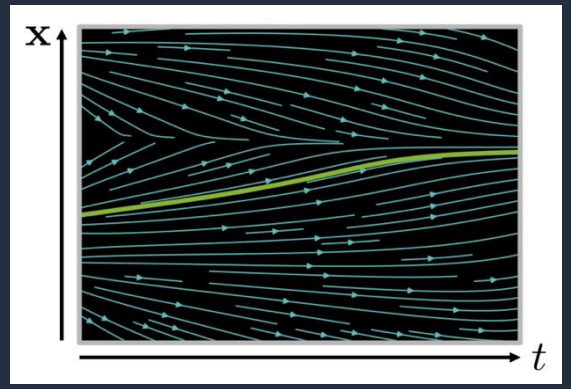
1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t\mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Score-based Modeling: Diffusion as SDE/ODE

Preliminaries: ODE/SDE

ODE: Ordinary differential equation

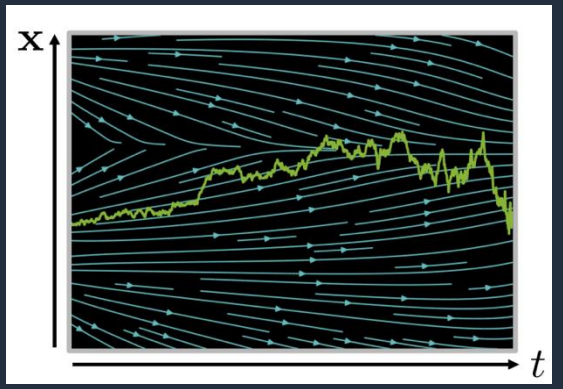
$$\frac{dx}{dt} = f(x, t)$$



Solution: $x(t) = x_0 + \int_o^t f(x(s), s)\, ds$

SDE: Stochastic differential equation

$$\frac{dx}{dt} = \underbrace{\mu(x, t)}_{\text{Drift term}} + \underbrace{\sigma(x, t)}_{\text{Diffusion term}} \underbrace{\omega_t}_{\text{Random Process}}$$



Solution: $x(t) = x_0 + \int_o^t \mu(x(s), s)\, ds + \int_0^t \sigma(x(s), s)\, d\omega_s$

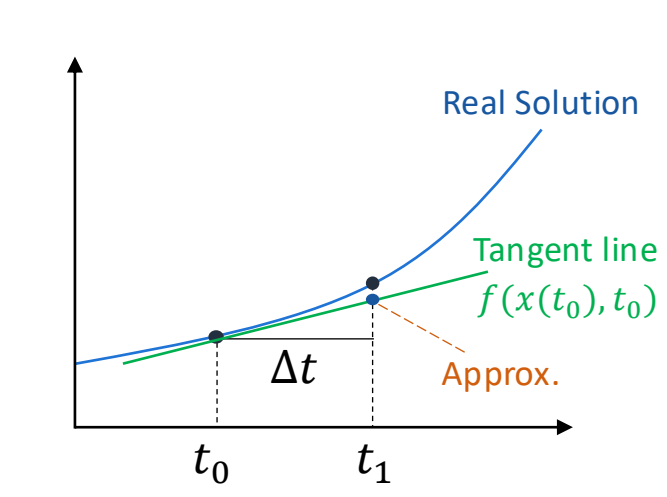
Image Credit: Copyright (c) 2023 cvpr2023-tutorial-diffusion-models (MIT license)

Preliminaries: Euler's method

- For ordinary differential equations, a typical solution can be written in the form of an integral.

$$\frac{dx}{dt} = f(x,t) \longrightarrow \text{Solution: } x(t) = x_0 + \int_o^t f(x(s),s)\,ds$$

- However, solving the integral can be challenging, so we often resort to approximation methods.



- We begin by discretizing the function into small steps and then use these to approximate the next value

$$x(t_{k+1}) = x(t_k) + f(x(t_k),t_k)\Delta t$$

-> This is the **Euler's method**

Preliminaries: Euler-Maruyama method

- Similarly, for stochastic differential equations, we should solve an integral to get the solution

$$\frac{dx}{dt} = \mu(x,t) + \sigma(x,t)\omega_t \longrightarrow \text{Solution: } x(t) = x_0 + \int_o^t \mu(x(s),s)\,ds + \int_0^t \sigma(x(s),s)\,d\omega_s$$

- For simplicity, we again discretize into small step size and then approximate

$$x(t_{k+1}) = x(t_k) + \mu(x(t_k),t_k)\Delta t + \sigma(x(t_k),t_k)\Delta\omega$$

ω_t is a Wiener process which $\Delta\omega \sim N(0,\Delta t I)$

Wiener process: models the random, continuous motion of a particle in Brownian motion.

$\omega(0) = 0$ $\omega(t+\Delta t) - \omega(t) = \Delta\omega \sim N(0,\Delta t I)$ Path is continuous (Almost surely) Independent Increment

Preliminaries: Fokker-Planck Equation

- In physics, the **Fokker-Planck equation** describes how the **probability density** of a particle's position evolves over time under stochastic dynamics.

$$\frac{dp(x,t)}{dt} = -\nabla\big(\mu(x,t)p(x,t)\big) + \nabla^2(\sigma(x,t)\cdot p(x,t))$$

$p(x,t)$ = probability density
$\mu(x,t)$ = drift term
$\sigma(x,t)$ = diffusion term

- Over time, the Fokker-Planck process may converge to a **stationary distribution** $\boldsymbol{p^*(x)}$

$$\frac{dp^*(x,t)}{dt} = 0$$

- In the machine learning perspective, we can think of this as designing a stochastic process such that its stationary distribution matches the data distribution $p_{data}(x)$

Preliminaries: Langevin dynamics

- **Langevin dynamics** describes the motion of a particle under both deterministic forces and random noise.
- We can use it to **simulate** samples from the **stationary distribution** of the Fokker–Planck equation.

- General Langevin equation:

$$dx(t) = -\nabla U\big(x(t)\big)dt + \sqrt{2D}\,d\omega(t)$$

- Corresponding Fokker-Planck:

$$\frac{dp(x,t)}{dt} = \nabla\Big(\nabla U\big(x(t)\big)p(x,t)\Big) + \nabla^2(D \cdot p(x,t))$$

Preliminaries: Langevin dynamics

- General Langevin equation: $dx(t) = -\nabla U\big(x(t)\big)dt + \sqrt{2D}d\omega(t)$
- Corresponding Fokker-Planck: $\frac{dp(x,t)}{dt} = \nabla\Big(\nabla U\big(x(t)\big)p(x,t)\Big) + \nabla^2(D \cdot p(x,t))$
- Stationary distribution:

$$\frac{dp(x,t)}{dt} = 0 \longrightarrow p^*(x) \propto \exp\left(\frac{-U(x)}{D}\right)$$

- If we choose $D = \frac{1}{2}, U(x) = -\frac{1}{2}\nabla \log p(x)$ $\longrightarrow$ $p^*(x) \propto \exp\left(\frac{\nabla \log p(x)\,/2}{1/2}\right) = p(x)$

Think of $p(x)$ as data distribution we want to target $\longrightarrow$ Stationary distribution matches our target distribution!

Preliminaries: Langevin dynamics

Generative SDE we wish to solve !

$$dx(t) = \frac{1}{2}\underbrace{\nabla \log p(x)\, dt}_{\text{Score function}} + \underbrace{d\omega(t)}_{\text{Noise}}$$

Score function = gradient of the log-probability density

= direction in which the density increases fastest at point x

Diffusion as SDE

Recall: DDPM forward

$$q(x_t|x_{t-1}) \coloneqq \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t \mathrm{I})$$

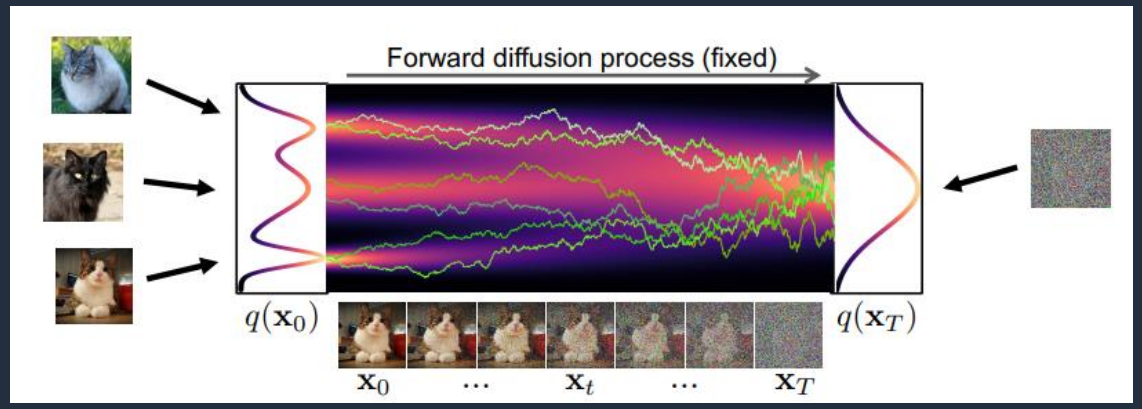
$$\longrightarrow \quad x_{t+1} = \sqrt{1-\beta_{t+1}}x_t + \sqrt{\beta_{t+1}}\epsilon$$

Let $t = \frac{i}{T} \in [0,1]$, Assume the step size $\Delta t = \frac{1}{T}$

$$\longrightarrow \quad x_{t+\Delta t} = \sqrt{1-\beta(t)\Delta t}x_t + \sqrt{\beta(t)}\epsilon$$

$$\longrightarrow \quad x_{t+\Delta t} = (1-\frac{1}{2}\beta(t)\Delta t)x_t + \sqrt{\beta(t)\Delta t}\epsilon$$

$$\longrightarrow \quad x_{t+\Delta t} - x_t = -\frac{1}{2}\beta(t)\Delta t x_t + \sqrt{\beta(t)\Delta t}\epsilon$$



(1st order Taylor expansion)

Image Credit: Copyright (c) 2023 cvpr2023-tutorial-diffusion-models (MIT license)

Diffusion as SDE

$$\longrightarrow \quad x_{t+\Delta t} - x_t = -\frac{1}{2}\beta(t)\Delta t x_t + \sqrt{\beta(t)\Delta t}\epsilon$$

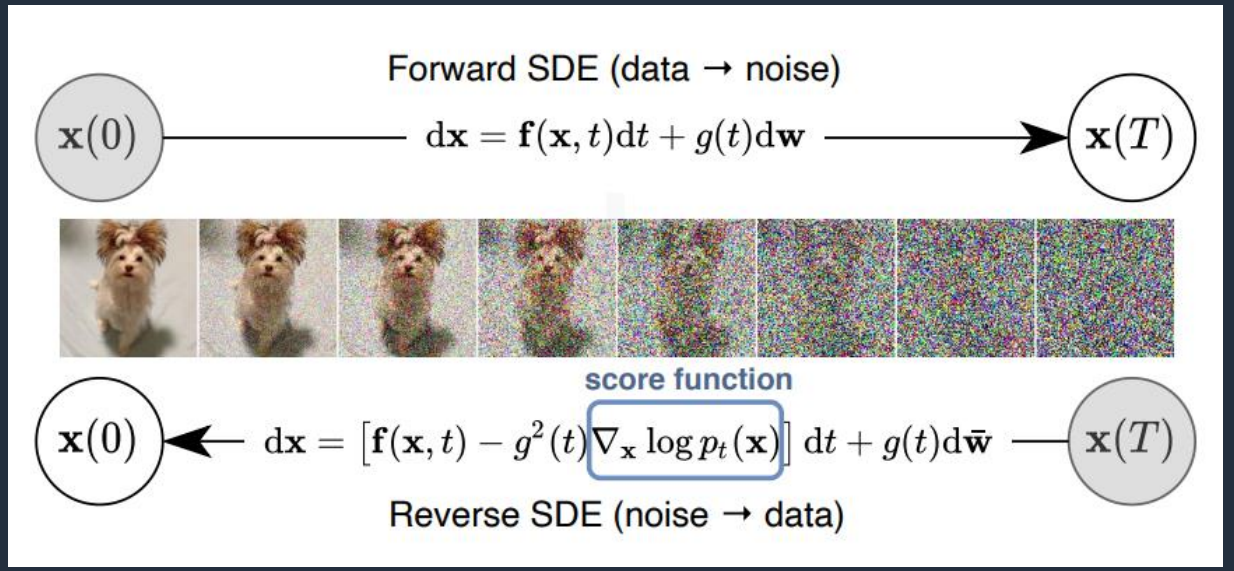
$$\longrightarrow \quad \boxed{dx(t) = -\frac{1}{2}\beta(t)x(t)dt + \sqrt{\beta(t)}d\omega(t)} \qquad \text{(Wiener process property)}$$

SDE for diffusion forward process

Diffusion as SDE

Forward SDE

$$dx(t) = -\frac{1}{2}\beta(t)x(t)dt + \sqrt{\beta(t)}d\omega(t)$$



Reverse SDE

$$dx(t) = [-\frac{1}{2}\beta(t)x(t) - \beta(t)\nabla_x \log q_t\big(x(t)\big)]dt + \sqrt{\beta(t)}d\bar{\omega}(t)$$

Score Function

Ref: Song et al., "SCORE-BASED GENERATIVE MODELING THROUGH STOCHASTIC DIFFERENTIAL EQUATIONS", ICLR 2021.

Score Matching

- Calculating the score function $\nabla_x \log q_t(x(t))$

This marginal is intractable

- Diffused $q_t(x(t)|x(0))$ is tractable

$$\min_{\boldsymbol{\theta}} \underbrace{\mathbb{E}_{t\sim\mathcal{U}(0,T)}}_{\text{diffusion time } t}\underbrace{\mathbb{E}_{\mathbf{x}_0\sim q_0(\mathbf{x}_0)}}_{\text{data sample } \mathbf{x}_0}\underbrace{\mathbb{E}_{\mathbf{x}_t\sim q_t(\mathbf{x}_t|\mathbf{x}_0)}}_{\text{diffused data sample } \mathbf{x}_t}||\underbrace{\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)}_{\text{neural network}} - \underbrace{\nabla_{\mathbf{x}_t}\log q_t(\mathbf{x}_t|\mathbf{x}_0)}_{\text{score of diffused data sample}}||_2^2$$

Slide Credit: Copyright (c) 2023 cvpr2023-tutorial-diffusion-models (MIT license)

Relation of DDPM with SDE

Prove: DDPM is the discrete time Euler-Maruyama approximation for above SDE.

- Reverse SDE

$$dx(t) = [-\frac{1}{2}\beta(t)x(t) - \beta(t)\nabla_x \log q_t\big(x(t)\big)]dt + \sqrt{\beta(t)}d\bar{\omega}(t)$$

- Discrete grid

$$t_k = \frac{k}{T}, \Delta t = t_k - t_{k-1} = \frac{1}{T}$$

- Euler-Maruyama discretization

$$x_{k-1} = x_k + [\frac{1}{2}\beta(t_k)x_k + \beta(t_k)\nabla_x \log q_t(x_k)]\Delta t + \sqrt{\beta(t_k)\Delta t}\epsilon_k$$

$$x_{k-1} = x_k + \frac{1}{2}\beta_k x_k + \beta_k \nabla_x \log q_t(x_k) + \sqrt{\beta_k}\epsilon_k$$

Relation of DDPM with SDE

$$x_{k-1} = x_k + \frac{1}{2}\beta_k x_k + \beta_k \nabla_x \log q_k(x_k) + \sqrt{\beta_k}\epsilon_k$$

- Let $\nabla_x \log q_k(x_k) = -\frac{1}{\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k)$

$$x_{k-1} = (1+\frac{1}{2}\beta_k)x_k - \beta_k \frac{1}{\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k) + \sqrt{\beta_k}\epsilon_k$$

- Recall DDPM reverse process

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \beta_t \frac{1}{\sqrt{1-\overline{\alpha_t}}}\epsilon_\theta(x,t) + \sqrt{\beta_t}\epsilon_t \quad \longrightarrow \quad \frac{1}{\sqrt{\alpha_t}} \approx 1 + \frac{1}{2}\beta_t$$ (1st order Taylor)

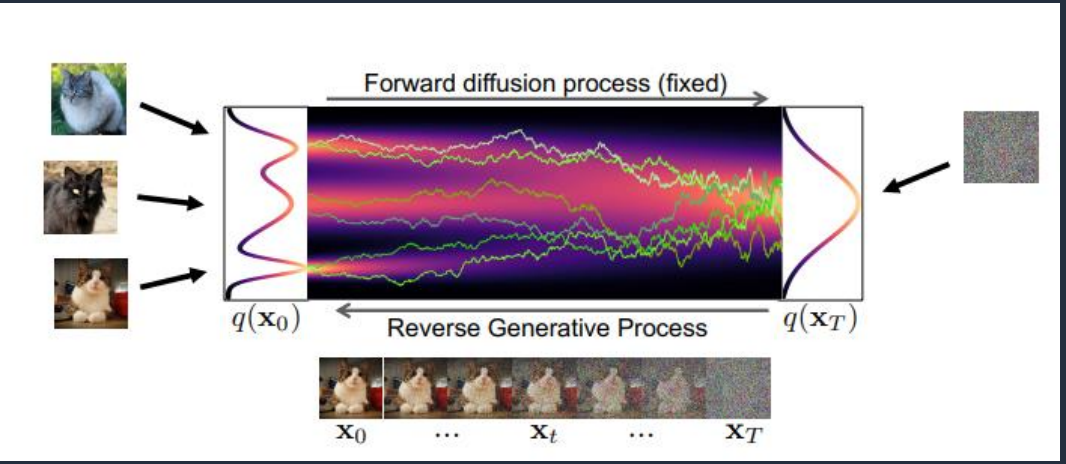
⟶ Euler-Maruyama approximation of SDE is same as the 1st order Taylor expansion of DDPM

Relation of DDIM with ODE

- Probability flow ODE

$$dx(t) = [-\frac{1}{2}\beta(t)x(t) - \frac{1}{2}\beta(t)\nabla_x \log q_t(x(t))]dt$$

⟶ This leads to same $q_t(x(t))$ with Reverse SDE



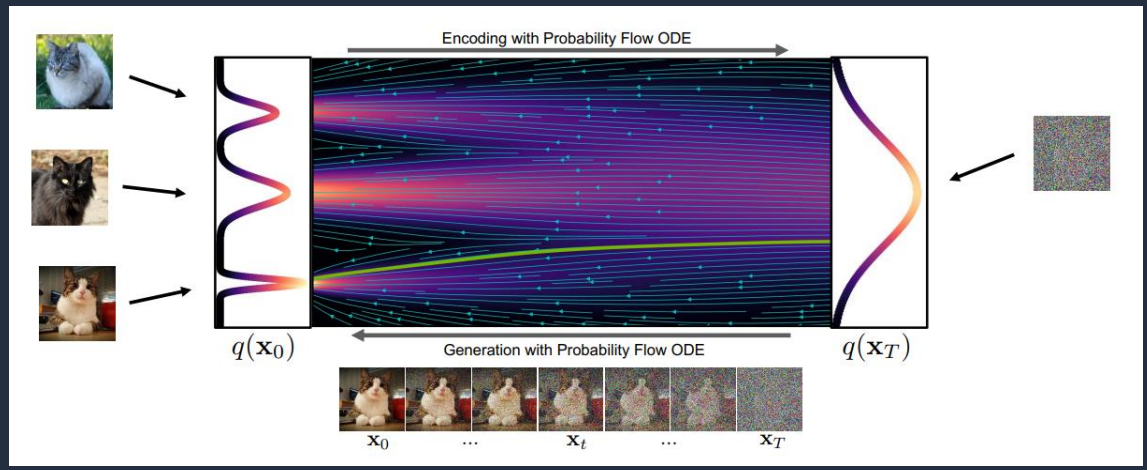


Image Credit: Copyright (c) 2023 cvpr2023-tutorial-diffusion-models (MIT license)

Relation of DDIM with ODE

Prove: DDIM is the discrete time Euler approximation for above ODE.

- Probability flow ODE

$$dx(t) = [-\frac{1}{2}\beta(t)x(t) - \frac{1}{2}\beta(t)\nabla_x \log q_t\big(x(t)\big)]dt$$

- Discrete grid

$$t_k = \frac{k}{T}, \Delta t = t_k - t_{k-1} = \frac{1}{T}$$

- Euler discretization

$$x_{k-1} = x_k - \frac{1}{2}[\beta(t_k)x_k + \beta(t_k)\nabla_x \log q_k(x_k)](-\Delta t)$$

$$x_{k-1} = x_k + \frac{1}{2}[\beta_k x_k + \beta_k \nabla_x \log q_k(x_k)]$$

Image Credit: Copyright (c) 2023 cvpr2023-tutorial-diffusion-models (MIT license)

Relation of DDIM with ODE

$$x_{k-1} = x_k + \frac{1}{2}[\beta_k x_k + \beta_k \nabla_x \log q_k(x_k)]$$

- Let $\nabla_x \log q_k(x_k) = -\frac{1}{\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k)$

$$x_{k-1} = x_k + \frac{1}{2}\left[\beta_k x_k - \beta_k \frac{1}{2\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k)\right] = x_k + \frac{1}{2}\beta_k\left[x_k - \frac{1}{2\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k)\right]$$

$$= \left(1 + \frac{1}{2}\beta_k\right) x_k - \frac{\beta_k}{2\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k)$$

$$\approx \frac{1}{\sqrt{\alpha_k}} x_k - \frac{\beta_k}{2\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k)$$

$$= \frac{1}{\sqrt{\alpha_k}}\left(x_k - \frac{\sqrt{\alpha_k}\beta_k}{2\sqrt{1-\overline{\alpha_k}}}\epsilon_\theta(x,k)\right) \longrightarrow$$

When plugging in x_0 formula, this leads to same DDIM reverse

Image Credit: Copyright (c) 2023 cvpr2023-tutorial-diffusion-models (MIT license)

Other SDE/ODE solvers could be utilized

- Euler method (ODE)
- Euler-Maruyama (SDE)
- Runge-Kutta (ODE)
- Heun's method (ODE)
- DPM-Solver
- …

DPM-Solver(Lu et al., 2022)

Algorithm 1 DPM-Solver-2.

Require: initial value $\boldsymbol{x}_T$, time steps $\{t_i\}_{i=0}^M$, model $\boldsymbol{\epsilon}_\theta$

$\tilde{\boldsymbol{x}}_{t_0} \leftarrow \boldsymbol{x}_T$

for $i \leftarrow 1$ to M **do**

$s_i \leftarrow t_\lambda\left(\frac{\lambda_{t_{i-1}}+\lambda_{t_i}}{2}\right)$

$\boldsymbol{u}_i \leftarrow \frac{\alpha_{s_i}}{\alpha_{t_{i-1}}}\tilde{\boldsymbol{x}}_{t_{i-1}} - \sigma_{s_i}\left(e^{\frac{h_i}{2}}-1\right)\boldsymbol{\epsilon}_\theta(\tilde{\boldsymbol{x}}_{t_{i-1}}, t_{i-1})$

$\tilde{\boldsymbol{x}}_{t_i} \leftarrow \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\tilde{\boldsymbol{x}}_{t_{i-1}} - \sigma_{t_i}\left(e^{h_i}-1\right)\boldsymbol{\epsilon}_\theta(\boldsymbol{u}_i, s_i)$

end for

return $\tilde{\boldsymbol{x}}_{t_M}$



Ref: Lu et al., "DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps", (NeurIPS 2022).

Next week

Week 6
Various diffusion spaces

- Diffusion models for **discrete space**
- Diffusion for **latent space**
- DiT (Diffusion transformers)

Week 7
Technical components

- Conditional generation (classifier guidance, CFG)
- Noise schedule
- Distillation, consistency models

Thank you!

AIDAS Lab